

INFORME TÉCNICO CONSOLIDADO: DIAGNÓSTICO DE FIABILIDAD (ISO/IEC 25010)

Equipo Evaluador: SQA Equipo 11
Proyecto Evaluado: Sistema de Gestión Bibliotecaria (Equipo 58-1)
Artefacto Analizado: Código Fuente y Repositorio de Solución (Java 21 / Spring Boot 3.4.5 / Vue 3.2.13)
Base Documental: Síntesis de Inspecciones Individuales Independientes
Instancia del Control: Dictamen y Cierre del Workflow 4 (WF4)

1. MEMORÁNDUM DEL PROCESO DE INSPECCIÓN COLECTIVA

Se realizó una compilación y contrastación de las observaciones técnicas derivadas de las inspecciones estáticas individuales efectuadas por los miembros del equipo sobre el repositorio de código. El proceso se centró en unificar el conocimiento colectivo respecto a la adherencia de las capas de software frente a los 11 criterios de control de Fiabilidad. A través de este análisis de revisión en frío, se unificaron las discrepancias de los reportes previos, se identificaron brechas estructurales severas compartidas en las dimensiones de backend, frontend y arquitectura de datos, y se fundamentó técnicamente el bloqueo del pipeline dentro del Workflow 4 (WF4).

2. COMPILACIÓN DE HALLAZGOS Y OBSERVACIONES DEL EQUIPO

CÓDIGO: COD-FIAB-01 DIMENSIÓN: BACKEND (JAVA) ¿El código captura excepciones específicas en operaciones críticas (IO, Parsing de fechas, DB) evitando bloques catch genéricos vacíos?
OBSERVACIONES CONSOLIDADAS DEL EQUIPO: Se detectó una vulnerabilidad crítica inicial en el archivo <code>PrestamoService.java</code> (Línea 47), donde se identificó un parseo directo con el método <code>LocalDate.parse(fechaPrestamoStr)</code> desprovisto de bloques de control de excepciones. El equipo evidenció que la omisión de un bloque <code>try-catch</code> específico ante errores de conversión (<code>DateTimeParseException</code>) generó un riesgo alto de interrupción abrupta del flujo del negocio en el backend.
CÓDIGO: COD-FIAB-02 DIMENSIÓN: BACKEND (JAVA) ¿Las llamadas síncronas a servicios externos o APIs de terceros implementan de forma explícita políticas de Timeout de conexión y lectura?
OBSERVACIONES CONSOLIDADAS DEL EQUIPO: Se examinaron las dependencias del backend y se constató que la aplicación no integró componentes de comunicación síncrona saliente tales como <code>RestTemplate</code> o <code>WebClient</code> para interacciones con APIs de terceros. Por consiguiente, el equipo determinó de forma unánime calificar este criterio bajo el estado de No Aplica dentro de los límites del incremento de código actual.

CÓDIGO: COD-FIAB-03 | DIMENSIÓN: TRANSVERSAL

¿Se utilizan patrones de resiliencia tipo Circuit Breaker o aislamiento ante fallos en acoplamientos remotos?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se buscó la presencia de bibliotecas de resiliencia distribuida (como Resilience4j). Tras verificar que la arquitectura se concibió para operar con dependencias locales y acoplamientos cerrados, el equipo concluyó que no existieron entornos remotos que justificaran la inyección de disyuntores, catalogando el ítem como No Aplica.

CÓDIGO: COD-FIAB-04 | DIMENSIÓN: BACKEND (JAVA)

¿El backend dispone de un manejador global de excepciones para interceptar errores imprevistos, evitando caídas completas del hilo y fugas de stacktrace?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se comprobó una ausencia severa en la infraestructura del backend al no localizarse clases controladoras anotadas con `@ControllerAdvice` o `@RestControllerAdvice`. El equipo determinó que esta carencia impidió la interceptación de fallos imprevistos en tiempo de ejecución, lo cual expuso al sistema a la caída del hilo de procesamiento y a la potencial filtración de trazas de pila detalladas hacia los clientes.

CÓDIGO: COD-FIAB-05 | DIMENSIÓN: FRONTEND (VUE)

En la interfaz de usuario (Vue 3), ¿se hace uso del hook de ciclo de vida `onErrorCaptured` o un manejador global de errores para encapsular fallos de renderizado?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se auditó el código del frontend y se observó que la aplicación no incorporó mecanismos de captura global de errores ni implementó el hook de ciclo de vida `onErrorCaptured` en los componentes raíz de Vue 3. Se verificó que las excepciones ocurridas en el árbol de renderizado reactivo carecieron de fronteras de contención, conllevando a la desconfiguración visual completa de la interfaz.

CÓDIGO: COD-FIAB-06 | DIMENSIÓN: TRANSVERSAL

¿Las operaciones asíncronas (@Async en Spring / Promises en Vue 3) poseen su respectivo manejo interno de fallos y rechazos?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se rastrearon los métodos de asincronía y el uso de promesas en la interfaz. El equipo identificó que las llamadas estructuradas para consumir la API omitieron de manera sistemática los bloques de captura final (`.catch()`) o las cláusulas de control en funciones asíncronas, ocasionando que las promesas rechazadas (unhandled promise rejections) quedaran desatendidas en el cliente.

CÓDIGO: COD-FIAB-07 | DIMENSIÓN: BACKEND (JAVA)

¿Todas las operaciones de mutación múltiple en el almacenamiento (persistencia) se ejecutan bajo un contexto transaccional atómico controlado?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se halló una deficiencia grave de consistencia en la capa de servicios del backend. El equipo constató que rutinas complejas de mutación —como el método `crearPrestamo`— alteraron múltiples entidades de almacenamiento simultáneamente (modificación en la tabla `Prestamo` y actualización de stock en la tabla `Libro`) sin incluir la anotación `@Transactional(rollbackFor = Exception.class)`, comprometiendo gravemente la atomicidad de la base de datos ante fallos intermedios.

CÓDIGO: COD-FIAB-08 | DIMENSIÓN: ARQUITECTURA DATOS

¿La configuración del Pool de Conexiones a la Base de Datos posee parámetros estáticos para restablecer enlaces caídos de forma segura?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se procedió a revisar las propiedades de configuración de persistencia del proyecto. Se evidenció que los archivos de propiedades de Spring Boot carecieron de directrices paramétricas específicas para regular el pool HikariCP (como la definición explícita de `connectionTimeout` o pruebas de validación de enlaces), lo cual impidió asegurar una restauración automática controlada de conexiones caídas.

CÓDIGO: COD-FIAB-09 | DIMENSIÓN: BACKEND (JAVA)

¿Las peticiones distribuidas propensas a fallas transitorias de red implementan políticas estáticas de reintento (Retry) con retroceso exponencial?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se confirmó la ausencia de arquitecturas distribuidas, llamadas remotas o el uso del módulo Spring Retry en los servicios integrados. En virtud de que las transacciones y consultas operaron directamente dentro del mismo contexto de ejecución local, el equipo ratificó de forma unánime el estado de No Aplica para este control.

CÓDIGO: COD-FIAB-10 | DIMENSIÓN: FRONTEND (VUE)

¿La interfaz frontend maneja estados de degradación elegante (Fallback) amigables frente a la indisponibilidad temporal del servidor API?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se verificó de manera concluyente un fallo en la lógica de la interfaz de usuario. Al auditar la capa de transporte del cliente HTTP, el equipo observó una omisión absoluta en la configuración de interceptores globales orientados a gestionar códigos de error de servidor (5xx). Esto provocó que la aplicación careciera de mecanismos de fallback o pantallas de degradación, gatillando el bloqueo absoluto de la interfaz ante caídas del servidor de API.

CÓDIGO: COD-FIAB-11 | DIMENSIÓN: ARQUITECTURA DATOS

En procesos batch o tareas programadas (@Scheduled), ¿existe un mecanismo de bitácora o persistencia de estado para reanudación segura ante interrupciones?

OBSERVACIONES CONSOLIDADAS DEL EQUIPO:

Se constató, mediante el rastreo integral de anotaciones en las clases de configuración, que el repositorio no dispone de ejecuciones temporizadas automáticas ni rutinas batch programadas en segundo plano. Al responder el sistema bajo un modelo exclusivamente transaccional reactivo, el equipo catalogó este ítem como No Aplica.

3. CONCLUSIONES GENERALES Y ESTADO DE ACEPTACIÓN TÉCNICA

A partir del análisis cruzado de las observaciones compiladas, el equipo determinó que el Sistema de Gestión Bibliotecaria del Equipo 58-1 no satisfizo las condiciones de fiabilidad estática requeridas bajo el estándar ISO/IEC 25010. Se constató la presencia generalizada de deficiencias severas de código en ambas dimensiones de la solución. La desprotección transaccional en la mutación de tablas correlativas y la carencia absoluta de una estrategia centralizada de excepciones en el backend se sumaron críticamente a la desatención de errores asíncronos identificada en el cliente reactivo Vue 3.